

Unit 1: Professional Software Development, Processes & Agile

Handwritten Student Notebook

Syllabus Topics Covered:

OFFICIAL SYLLABUS TOPICS COVERED Professional Software Development: Software engineering definition, attributes of good software (maintainability, dependability, efficiency, acceptability), engineering diversity. Software Engineering Ethics: ACM/IEEE Code of Ethics & princi...

Table of Contents

Professional Software Development & Essential Attributes	p. 2
Software Engineering Ethics (ACM/IEEE Code)	p. 3
Three Landmark Case Studies (Sommerville)	p. 4
Software Process Models	p. 5
Coping with Change & Boehm's Spiral Model	p. 7
Process Improvement & The CMMI Framework	p. 9
Agile Software Development & Extreme Programming (XP)	p. 10
The Scrum Framework, Sprint Velocity & Burndown	p. 11
One-Page Rapid Revision Sheet	p. 13
Important Examination Questions	p. 14
Authentic Solved Questions from Past Ramaiah CIE & SEE Papers	p. 16
Verified High-Yield Practice Problems	p. 21

Professional Software Development & Essential Attributes

§ 1

Software Engineering is an engineering discipline concerned with all aspects of software production from early system specification through to maintaining the system after it has gone into use.

SOFTWARE ENGINEERING VS COMPUTER SCIENCE:

- **Computer Science:** Concerned with theories, mathematical algorithms, and foundational computation principles.
- **Software Engineering:** Concerned with the practical problems of producing software: economics, project management, customer collaboration, team development, toolchains, and quality assurance.

The Four Essential Attributes of Good Software

Attribute	Engineering Definition	Practical Significance
1. Maintainability	Software must be written in such a way that it can evolve to meet the changing needs of customers.	Over 70% of lifecycle software costs occur after initial delivery during maintenance and adaptation.
2. Dependability & Security	Includes reliability, security, safety, and resilience. Software should not cause physical or economic damage in the event of system failure.	Essential for safety-critical medical devices, banking transactions, and aerospace control systems.
3. Efficiency	Software should not make wasteful use of system resources (memory, CPU processing cycles, network bandwidth).	Critical for battery-powered mobile devices and cloud microservices where compute costs scale with usage.

Attribute	Engineering Definition	Practical Significance
4. Acceptability	Software must be acceptable to the type of users for which it was designed: understandable, usable, and compatible with other systems.	Software with great architecture will still fail in the marketplace if the user experience is hostile.

QUICK RECALL

- Software engineering focuses on practical, cost-effective delivery of reliable software.
- 4 Essential Attributes: **Maintainability, Dependability/Security, Efficiency, Acceptability.**
- Maintenance constitutes the majority of the lifetime cost of software.

Software Engineering Ethics (ACM/IEEE Code)

§ 2

Software engineers do not simply apply technical skills; they carry ethical and social responsibilities. Software can directly impact human safety, privacy, and economic well-being. The joint **ACM/IEEE-CS Code of Ethics** establishes eight ethical principles.

Principle #	Principle Name	Core Ethical Mandate
1	PUBLIC	Software engineers shall act consistently with the public interest, prioritizing safety, health, and welfare.
2	CLIENT & EMPLOYER	Act in the best interests of their client and employer, consistent with the public interest.
3	PRODUCT	Ensure that products and related modifications meet the highest professional standards possible.
4	JUDGMENT	Maintain integrity and independence in their professional judgment; reject bribes or compromised audits.

Principle #	Principle Name	Core Ethical Mandate
5	MANAGEMENT	Managers and leaders shall subscribe to and promote an ethical approach to the management of software development.
6	PROFESSION	Advance the integrity and reputation of the profession consistent with the public interest.
7	COLLEAGUES	Be fair to and supportive of their colleagues; mentor junior engineers and credit intellectual contributions.
8	SELF	Participate in lifelong learning regarding the practice of their profession and promote an ethical approach.

QUICK RECALL

- ACM/IEEE Code of Ethics contains 8 principles.
- Top priority: **Public interest ALWAYS supersedes client or employer demands!**
- Confidentiality, Competence, and Intellectual Property must be honored.

Three Landmark Case Studies (Sommerville)

§ 3

Sommerville grounds software engineering principles in three diverse real-world systems, each illustrating different operational challenges.

Case Study System	System Domain	Core Architectural & Engineering Challenges
1. Insulin Pump Control System	Embedded safety-critical medical device	Monitors blood glucose levels and injects calculated micro-doses of insulin. Zero margin for error: overdosing induces hypoglycemic coma/death; underdosing causes organ failure. Requires formal verification, fault-tolerant fail-safe hardware backups, and rigorous plan-driven documentation.
2. Mentcare Mental Health System	Information system / Clinical record database	Maintains patient history and prescribed medications across regional mental health clinics. Primary challenge: balancing medical staff access in psychiatric emergencies with strict patient data privacy regulations (HIPAA/GDPR).
3. Wilderness Weather Station	Distributed sensor data collection	Autonomous weather sensor stations deployed in harsh, remote wilderness with solar/battery power and satellite communications. Primary challenge: power conservation, self-healing reboot mechanisms, dynamic reconfiguration, and autonomous sensor calibration without human intervention.

QUICK RECALL

- Insulin Pump: Safety-critical, formal verification, fail-safe architecture.
- Mentcare: Information system, privacy regulations vs emergency usability.
- Wilderness Weather Station: Distributed, low power, autonomous fault recovery.

Software Process Models

§ 4

A **software process** is a structured set of activities required to develop a software system. All processes involve four fundamental activities: **Specification** (defining requirements), **Development** (design and coding), **Validation** (testing), and **Evolution** (maintenance).

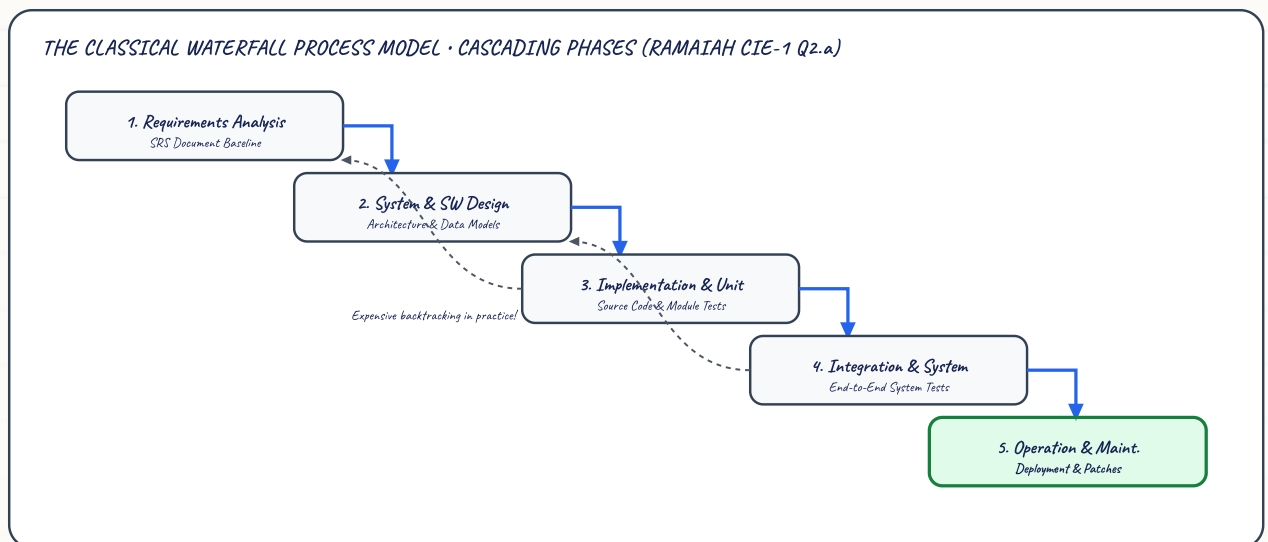


Figure 1.1: The Classical Waterfall Process Model with Downward Cascades (Sommerville · Ramaiah CIE-1 Q2.a)

Phases flow strictly in sequence: Requirements → System & SW Design → Implementation & Unit Testing → Integration & System Testing → Operations & Maintenance. Backtracking is cost-prohibitive in practice.

1. The Waterfall Model (Classic SDLC)

A strictly plan-driven, sequential process where each phase must be signed off before the next phase begins:

1

Requirements Analysis & Definition: System services, constraints, and goals established with users.

2

System & Software Design: Overall system architecture and data structures established.

3

Implementation & Unit Testing: Code written and verified as individual modules.

4

Integration & System Testing: Individual program units integrated and tested as a complete system.

5

Operation & Maintenance: System installed, put into practical use, and bugs fixed over time.

Strengths: Highly structured, predictable documentation, clear milestones for contract management.

Weaknesses: Inflexible partitioning; late integration; disaster strikes if requirements change halfway through.

2. Incremental Development

Specification, development, and validation are interleaved. The system is developed as a series of increments (versions), with each increment adding functional features to previous releases.

- **Advantages:** Cost of accommodating changing customer requirements is greatly reduced; early feedback from working software; rapid delivery of useful functionality.
- **Disadvantages:** Process is not visible (managers need documents to measure progress); system architecture tends to degrade as new increments are added unless refactored.

3. Reuse-Oriented Software Engineering (Integration & Configuration)

Relies on integrating existing Commercial Off-The-Shelf (COTS) software components, open-source libraries, or cloud web services rather than writing from scratch.

QUICK RECALL

- 4 Activities: Specification → Development → Validation → Evolution.
- Waterfall: sequential, rigid, document-driven, best for stable requirements (embedded/safety-critical).
- Incremental: interleaved, early feedback, lower risk of customer rejection.

Coping with Change & Boehm's Spiral Model

§ 5

Change is an inevitable reality in all software development. Software engineering processes incorporate two primary strategies to reduce the cost of rework: **System Prototyping** and **Boehm's Spiral Model**.

SYSTEM PROTOTYPING: THROWAWAY VS EVOLUTIONARY:

- **Throwaway Prototyping:** A quick prototype developed solely to clarify poorly understood requirements. Once requirements are clear, the prototype is discarded and the production system is engineered properly.
- **Evolutionary Prototyping:** Starts with an initial working prototype that is repeatedly refined through customer feedback, eventually evolving into the final delivered product.

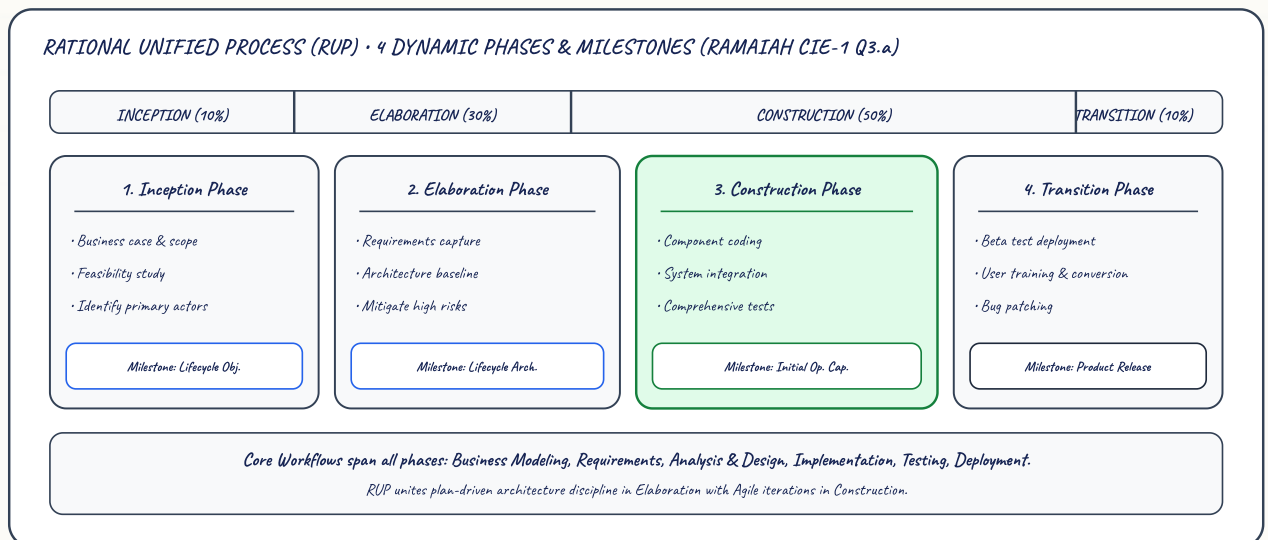


Figure 1.2: Rational Unified Process (RUP) 4 Distinct Phases & Exit Milestones (Ramaiah CIE-1 Q3.a)

Inception (Lifecycle Objective milestone) → Elaboration (Lifecycle Architecture milestone) → Construction (Initial Operational Capability milestone) → Transition (Product Release milestone).

Boehm's Spiral Model (Risk-Driven Process)

In Barry Boehm's Spiral Model, the software process is represented as a spiral rather than a sequence of activities with backtracking. Each loop represents a phase of the process, partitioned into **four distinct sectors**:

Sector #	Spiral Sector Name	Activities Conducted in Sector
Quadrant 1	Determine Objectives & Constraints	Specific objectives for the phase identified; constraints on the process and product determined; alternative management plans drawn up.
Quadrant 2	Identify & Resolve Risks	Detailed risk analysis conducted; prototyping and simulation used to resolve identified technical, management, and cost risks.

Sector #	Spiral Sector Name	Activities Conducted in Sector
Quadrant 3	Development & Testing	A development model (Waterfall, Incremental) chosen based on remaining risks; software is designed, coded, and verified.
Quadrant 4	Plan the Next Phase	Project is reviewed; decision made whether to continue with a further loop; next phase plans developed.

QUICK RECALL

- Spiral model is strictly **risk-driven**: each loop incorporates risk assessment.
- 4 Quadrants: (1) Objectives → (2) Risk Analysis → (3) Development → (4) Planning next phase.
- Radius of spiral represents cumulative cost; angular dimension represents progress.

Process Improvement & The CMMI Framework

§ 6

Process improvement aims to understand existing processes and change them to improve product quality, reduce costs, or accelerate delivery schedules. The **Capability Maturity Model Integration (CMMI)** provides a staged architecture to evaluate organizational maturity.

Level #	CMMI Maturity Level	Characterization of Organizational Process
Level 1	Initial (Ad-hoc)	Processes are unpredictable, poorly controlled, and reactive. Success depends entirely on individual heroics.
Level 2	Managed (Repeatable)	Basic project management practices established (tracking costs, schedule, functionality). Repeatable across similar projects.
Level 3	Defined	Processes documented, standardized, and integrated across the entire organization. Standard software process defined.

Level #	CMMI Maturity Level	Characterization of Organizational Process
Level 4	Quantitatively Managed	Detailed quantitative metrics collected for both process and product quality. Statistical process control applied.
Level 5	Optimizing	Continuous process improvement achieved through quantitative feedback and innovative pilot technology insertion.

QUICK RECALL

- CMMI 5 Levels: **Initial** → **Managed** → **Defined** → **Quantitatively Managed** → **Optimizing**.
- Level 2 establishes project tracking; Level 3 standardizes company-wide; Level 4 measures with stats; Level 5 optimizes continuously.

Agile Software Development & Extreme Programming (XP)

§ 7

Dissatisfaction with the overhead of heavy plan-driven models led in 2001 to the **Agile Manifesto**, prioritizing working software, collaboration, and rapid responsiveness over rigid process documentation.

THE 4 CORE VALUES OF THE AGILE MANIFESTO:

1. **Individuals and interactions** over processes and tools.
2. **Working software** over comprehensive documentation.
3. **Customer collaboration** over contract negotiation.
4. **Responding to change** over following a plan.

Extreme Programming (XP) Key Practices

- **Pair Programming:** Developers work in pairs at a single workstation (driver and navigator). Continuous peer code review improves quality and shares knowledge.
- **Test-First Development (TDD):** Automated unit tests are written before writing the code itself. Code is only considered complete when it passes all tests.
- **Refactoring:** Programmers continuously look for improvements to code structure ("cleaning code smells") even if no new functionality is added.

- **Continuous Integration:** Code is integrated into the mainline repository multiple times a day; automated tests immediately verify build integrity.
- **Small Releases:** Releases occur every two to three weeks, keeping user feedback immediate.

QUICK RECALL

- Agile values working software and responding to change over rigid documentation.
- XP practices: Pair Programming, Test-First Development (TDD), Refactoring, Continuous Integration.

The Scrum Framework, Sprint Velocity & Burndown

§ 8

Scrum is an agile framework that focuses on managing iterative development rather than specific programming practices. It provides a lightweight structure of roles, events, and artifacts.

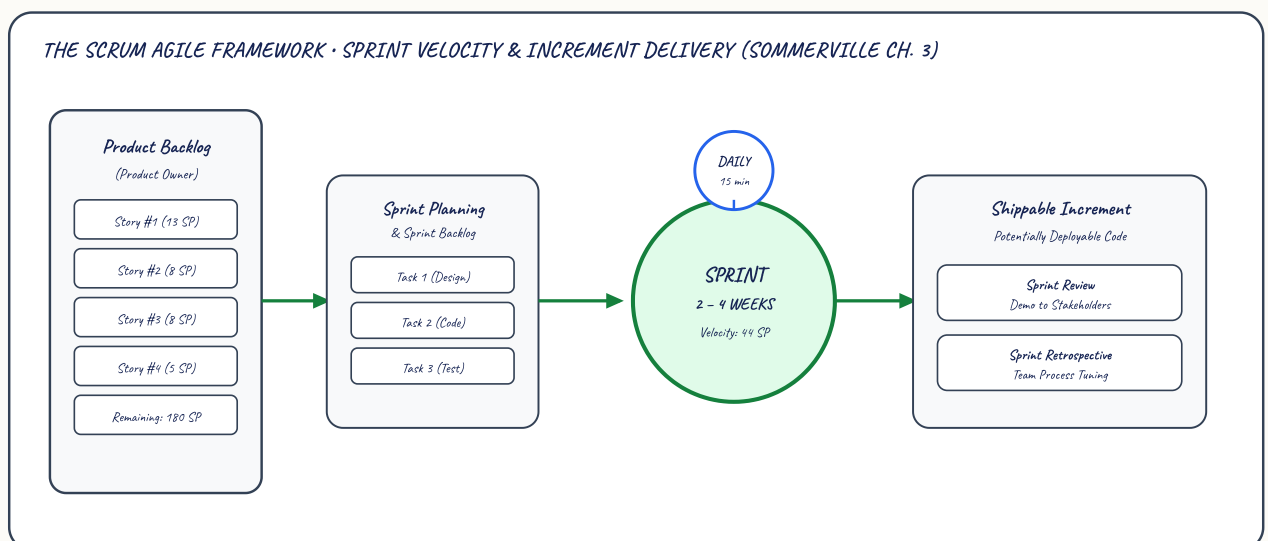


Figure 1.3: The Scrum Agile Framework Lifecycle · Sprints, Velocity & Increments (Sommerville Ch. 3)

Product Backlog → Sprint Planning → 2-4 Week Sprint Loop with 15-min Daily Standup → Potentially Shippable Increment with Sprint Review & Retrospective.

Scrum Roles, Events, and Artifacts

Scrum Dimension	Element	Role & Responsibility
Roles	Product Owner	Represents the business and customer. Owns, prioritizes, and maintains the Product Backlog.
	Scrum Master	Servant-leader. Ensures Scrum practices are followed, facilitates meetings, and removes external impediments for the team.
	Development Team	Self-organizing, cross-functional team (typically 5–9 people) that designs, develops, tests, and delivers the increment.
Events	Sprint	Fixed time-box of 1 to 4 weeks during which a "Done" usable increment is created.
	Sprint Planning	Team selects user stories from Product Backlog and defines how they will be built.
	Daily Scrum	15-minute daily standup answering: What did I do yesterday? What will I do today? Any blockers?
	Sprint Review	Demo of completed increment to stakeholders for live feedback.
	Sprint Retrospective	Team reflects on its own process to identify continuous improvements for next sprint.
Artifacts	Product Backlog	Prioritized master list of all features, enhancements, and bug fixes for the product.
	Sprint Backlog	The subset of product backlog items chosen for the current sprint plus technical tasks.
	Increment	The sum of all Product Backlog items completed during a Sprint meeting the Definition of Done.

WORKED NUMERICAL · FULLY VERIFIED PASS [8 MARKS · EXAM CORE]**Scrum Velocity and Release Burndown Calculation**

A software product backlog starts with a total estimated scope of **180 Story Points (SP)**. A Scrum team completes three consecutive 2-week sprints with the following completed story points:

- Sprint 1: Planned 45 SP → Completed 40 SP
- Sprint 2: Planned 45 SP → Completed 45 SP
- Sprint 3: Planned 50 SP → Completed 47 SP

Calculate: (a) The total completed points, (b) The team's average velocity, (c) The remaining backlog, and (d) The estimated number of additional sprints needed to finish the release.

Step 1: Total Completed Points:

$$\text{Total Completed} = 40 + 45 + 47 = \mathbf{132 \text{ Story Points}}$$

Step 2: Team Average Velocity:

$$\text{Velocity} = \frac{\text{Total Completed}}{\text{Number of Sprints}} = \frac{132}{3} = \mathbf{44.00 \text{ SP per sprint}}$$

Step 3: Remaining Backlog:

$$\text{Remaining Backlog} = 180 - 132 = \mathbf{48 \text{ Story Points}}$$

Step 4: Estimated Additional Sprints:

$$\text{Sprints Remaining} = \frac{\text{Remaining Backlog}}{\text{Average Velocity}} = \frac{48}{44.00} \approx \mathbf{1.09 \text{ sprint}}$$

The project requires $\lceil 1.09 \rceil = \mathbf{2}$ additional sprints (4 weeks) to fully deliver the remaining backlog items.

QUICK RECALL

- Scrum Roles: Product Owner (backlog priority), Scrum Master (coach/remover of blockers), Team (delivery).
- Scrum Events: Sprint, Planning, Daily Standup (15 min), Review (demo), Retrospective (process improvement).
- Velocity = Story points completed per sprint; Burndown tracks remaining work over time.

One-Page Rapid Revision Sheet

★ REVISION

Good Software Attributes

- Maintainability: evolvable to change (70% cost).
- Dependability/Security: reliable and safe.
- Efficiency: zero resource waste.
- Acceptability: usable and adopted by users.

Process Models

- Waterfall: sequential, plan-driven, document-heavy.
- Incremental: interleaved, early delivery, reduced risk.
- Spiral Model: 4 sectors, strictly risk-driven.
- CMMI: Initial → Managed → Defined → Quant Managed → Optimizing.

Extreme Programming (XP)

- Pair programming: driver & navigator.
- TDD: write automated unit tests before code.
- Refactoring: clean architecture continuously.
- Continuous integration: merge code multiple times/day.

Scrum Framework

- PO: owns product backlog. SM: process facilitator.
- Sprint: 1–4 weeks time-box.
- Artifacts: Product Backlog, Sprint Backlog, Increment.
- Velocity: $\sum SP_{\text{completed}} / \text{Sprints}$.

 **Interactive Tool Available Online:** ⚡ Interactive Scrum Sprint Velocity & Burndown Calculator. Explore the step-by-step calculator and visualizer on the live site notes page.

Important Examination Questions

? QUESTIONS

2-Mark Questions (Short Answer)

Practice Problem:

2M List the four essential attributes of professional software.

Solution:

Answer: (1) Maintainability, (2) Dependability and Security, (3) Efficiency, and (4) Acceptability.

Practice Problem:

2M What is the primary difference between plan-driven and agile development?

Solution:

Answer: In plan-driven development, all process activities are planned in advance and progress is measured against this formal plan. In agile development, planning is incremental and continual, and it is easier to change the software to meet evolving customer needs.

Practice Problem:

2M What are the four sectors in Boehm's Spiral Model?

Solution:

Answer: (1) Determine objectives and constraints, (2) Identify and resolve risks, (3) Development and validation, and (4) Plan the next phase.

5-Mark Questions (Medium Length)

Practice Problem:

SM Explain the roles and ceremonies in the Scrum framework.

Solution:

Answer: Detail the three roles (Product Owner, Scrum Master, Development Team) and the five standard ceremonies (Sprint, Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) as presented in Section 8.

Practice Problem:

SM Describe the five maturity levels of the CMMI framework.

Solution:

Answer: (1) Initial (ad-hoc), (2) Managed (project tracking), (3) Defined (organization-wide standards), (4) Quantitatively Managed (statistical process control), and (5) Optimizing (continuous technological innovation), as detailed in Section 6.

10-Mark Questions (Long Answer / Comprehensive)

Practice Problem:

10M Compare the Waterfall model with Incremental development. Under what project conditions should each be selected?

Solution:

Answer Summary: Compare both models across phase overlap, accommodation of requirement changes, customer feedback speed, documentation overhead, and architectural integrity. Recommend Waterfall for critical systems with fixed specifications (e.g. avionics, medical pumps) and Incremental for e-commerce, cloud software, and business applications with volatile requirements.

Authentic Solved Questions from Past Ramaiah CIE & SEE Papers

EXAM ARCHIVE

The following questions are transcribed directly from authentic internal assessment and semester-end examination papers in notes/se/practice/se-cie-1-and-2.pdf (Ramaiah Continuous Internal Evaluation - 1, October 28, 2025, Course Code 23IS51):

Dimension	Programming	Software Engineering
Scope & Lifecycle	Writing code to solve an immediate algorithmic task.	Encompasses requirements, architecture, testing, deployment, and decades of maintenance.
Scale & Team	Often individual or small team; single codebase.	Large distributed teams; hundreds of components; versioning and configuration control.
Quality Attributes	Focuses on functional correctness and algorithm speed.	Prioritizes maintainability, dependability, security, usability, and regulatory compliance.
Economic Impact	Development cost dominated by initial authoring.	Over 70% of total lifetime cost occurs during software evolution and maintenance!

Phase #	Phase Name	Key Engineering Activities & Tangible Deliverables
1	Requirements Analysis & Definition	System services, constraints, and operational goals are established through customer consultation. <i>Deliverable:</i> Formally signed-off Software Requirements Specification (SRS) document.
2	System & Software Design	Partitions requirements into hardware and software subsystems. Establishes overall architectural frameworks, data structures, and interface definitions. <i>Deliverable:</i> Software Architecture & High-Level Design Document.

Phase #	Phase Name	Key Engineering Activities & Tangible Deliverables
3	Implementation & Unit Testing	Software design is realized as a set of program modules. Individual units are coded and verified against module specifications. <i>Deliverable:</i> Verified source code units and unit test suites.
4	Integration & System Testing	Individual program units are integrated and tested as a unified software system to ensure complete compliance with the SRS. <i>Deliverable:</i> Verified integrated system and test sign-off report.
5	Operation & Maintenance	System is deployed into production. Maintenance involves fixing undiscovered errors, adapting software to environmental changes, and adding new requirements. <i>Deliverable:</i> Live software patches and version releases.

RUP Phase	Primary Objectives & Focus	Key Exit Milestone
1. Inception	Establish the business case for the system. Identify all external entities (people and systems) that will interact with the system. Define all key use cases and estimate resource/schedule feasibility.	Lifecycle Objective (LCO) Milestone: Stakeholders agree on scope, cost estimates, and whether to fund the project.
2. Elaboration	Develop an understanding of the problem domain. Establish an architectural baseline for the system. Identify the highest-risk technical elements and resolve them through executable prototypes.	Lifecycle Architecture (LCA) Milestone: A stable, verified architecture is established. High risks mitigated.

RUP Phase	Primary Objectives & Focus	Key Exit Milestone
3. Construction	Focuses on component development, coding, and integration. The remaining system features are designed, implemented, and unit tested. System integration testing is conducted across iterative increments.	Initial Operational Capability (IOC) Milestone: A working beta release is ready for initial user trials.
4. Transition	Moving the software from development into production use. Conducting beta testing, user training, data conversion, and defect corrections. Ensuring customer satisfaction before formal handover.	Product Release Milestone: The software system is officially handed over to the client.

Ramaiah SEE Archive · Model Question Paper [8 Marks]

"Explain Extreme Programming (XP) practices including Test-Driven Development (TDD), Pair Programming, Refactoring, and Small Releases."

 Practice Problem:

Show Complete Step-by-Step Model Answer

Solution:

Extreme Programming (XP) (Kent Beck) pushes recognized software engineering best practices to extreme levels:

- **1. Test-Driven Development (TDD):** Tests are written *before* functional code. The developer writes an automated unit test for a new requirement, watches it fail, writes the minimal code necessary to make it pass, and then refactors. TDD guarantees comprehensive test coverage and prevents regression defects.
- **2. Pair Programming:** Two programmers work together at a single workstation. One acts as the *Driver* (typing code), while the other acts as the *Navigator* (reviewing code in real time, checking for edge cases and design adherence). Roles switch frequently, improving collective ownership and code quality.
- **3. Refactoring:** Continuous code restructuring without altering observable external behavior. Programmers proactively clean up duplication, simplify complex methods, and eliminate code smells. Refactoring preserves architectural maintainability across rapid iterations.
- **4. Small Releases:** Working software increments are released to real users every 2 to 3 weeks. Rapid releases yield immediate customer feedback and drastically reduce financial risk.

<i>Evaluation Dimension</i>	<i>Plan-Driven Development (e.g., Waterfall)</i>	<i>Agile Development (e.g., Scrum / XP)</i>
Customer Involvement	Customer involved primarily at beginning (requirements sign-off) and end (acceptance testing).	Customer is continuously engaged (e.g., on-site customer or active Product Owner in every sprint).
Response to Change	Formal Change Control Board (CCB); changes are resisted as costly disruptions to the plan.	Welcomes changing requirements at any stage as a competitive advantage for the client.
Team Structure	Hierarchical, specialized roles (analysts, designers, coders, testers).	Cross-functional, self-organizing, collaborative teams.
Documentation Standard	Comprehensive formal documents required at every phase boundary (SRS, SDD, Test Plans).	Minimal documentation; working executable code and automated unit tests serve as ground truth.

Verified High-Yield Practice Problems

▽ PRACTICE

Test your software engineering reasoning with these exam-caliber problems. All numerical calculations are verified against `audit/verify/se/verify_se_u1.py`:

PRACTICE 1 · SPRINT VELOCITY & RELEASE ESTIMATION

A software project has an initial product backlog of 180 Story Points (SP). Over three 2-week sprints, the team completes 40 SP, 45 SP, and 47 SP. Calculate the average velocity, remaining backlog, and projected sprints to completion.

 Practice Problem:

Show Step-by-Step Solution

Solution:**1. Total Story Points Completed:**

$$\text{Total Completed} = 40 + 45 + 47 = 132 \text{ SP}$$

2. Average Team Velocity:

$$V_{\text{avg}} = \frac{132}{3} = \mathbf{44.00 \text{ SP/sprint}}$$

3. Remaining Product Backlog:

$$\text{Remaining Scope} = 180 - 132 = 48 \text{ SP}$$

4. Projected Sprints Remaining:

$$\text{Sprints to Completion} = \frac{48}{44.00} = \mathbf{1.09 \text{ sprints}}$$

$\implies \mathbf{2 \text{ sprints}}$

Conclusion: The team will deliver the remaining scope comfortably in approximately 2 further 2-week sprints (4 weeks of calendar time).

PRACTICE 2 · SAFETY-CRITICAL VS STARTUP PROCESS SELECTION

Which process model would you recommend for: (a) An embedded automated insulin pump controller, (b) A consumer photo-sharing mobile application? Justify your choice based on engineering characteristics.

Practice Problem:

Show Step-by-Step Solution

Solution:

(a) Automated Insulin Pump Controller:

- **Recommendation:** Plan-Driven Waterfall / V-Model with Formal Verification.
- **Justification:** Safety-critical embedded system where software bugs can cause human fatality. Requirements (sensor limits, insulin delivery rates) must be frozen and thoroughly analyzed upfront. Regulatory approval (FDA/CE) requires rigorous, traceable documentation connecting every requirement to verification proofs. Agile's "move fast and refactor" is completely unacceptable here.

(b) Consumer Photo-Sharing Mobile Application:

- **Recommendation:** Agile Development (Scrum / Kanban).
- **Justification:** Fast-moving consumer market where user preferences are volatile and time-to-market is critical. Rapid 2-week release cycles allow the company to test new filters and social features with real users and adapt based on analytics.

PRACTICE 3 · ACM/IEEE CODE OF ETHICS ANALYSIS

A project manager pressures a software engineer to sign off on an untested safety module for an autonomous drone to meet a contractual delivery deadline. Which ACM/IEEE ethical principles are violated? What should the engineer do?

Practice Problem:

Show Step-by-Step Solution

Solution:

1. Principles Violated:

- **Principle 1 (PUBLIC):** Software engineers must prioritize public safety, health, and welfare above all commercial considerations. Signing off on untested safety software endangers the public.
- **Principle 3 (PRODUCT):** Requires engineers to ensure products meet the highest professional standards and undergo proper testing.
- **Principle 4 (JUDGMENT):** Mandates that engineers maintain professional independence and refuse to sign off on documents they cannot verify.
- **Principle 5 (MANAGEMENT):** Managers must promote ethical software engineering rather than cut corners to meet artificial deadlines.

2. Required Engineering Action: The engineer must refuse to sign off, document the test gaps in writing to management, and escalate to higher authority if the pressure persists.

PRACTICE 4 · CMMI CAPABILITY MATURITY PROGRESSION

Explain the five maturity levels of the Capability Maturity Model Integration (CMMI) and identify the key milestone that distinguishes Level 3 from Level 2.

Practice Problem:

Show Step-by-Step Solution

Solution:

1. **Level 1 (Initial)**: Ad-hoc, chaotic processes; success depends on individual heroics.
2. **Level 2 (Managed)**: Basic project management processes established at the project level (cost, schedule, requirements tracking). Repeatable across similar projects.
3. **Level 3 (Defined)**: Processes are documented, standardized, and integrated across the **entire organization**. Standard training and organization-wide processes are enforced.
4. **Level 4 (Quantitatively Managed)**: Detailed quantitative metrics and statistical process control techniques are applied to measure process performance and product quality.
5. **Level 5 (Optimizing)**: Continuous process improvement driven by quantitative feedback and pilot technology adoption.

Key Distinction (Level 2 vs Level 3): At Level 2, processes are managed within **individual project silos**; at Level 3, processes are **standardized company-wide** so lessons learned in one project automatically benefit the whole organization.

PRACTICE 5 · TEST-DRIVEN DEVELOPMENT (RED-GREEN-REFACTOR)

Explain the three stages of the TDD "Red-Green-Refactor" cycle and list two concrete engineering benefits of this methodology.

 Practice Problem:

Show Step-by-Step Solution

Solution:

1. **RED (Write a Failing Test):** Before writing any implementation code, write an automated unit test that specifies a small desired behavior. Run the test harness and confirm that the test fails (red bar).
2. **GREEN (Make the Test Pass):** Write the minimal amount of code necessary to make the test pass (green bar). Elegance is secondary at this step; focus purely on passing the test.
3. **REFACTOR (Clean the Code):** Clean up duplicate code, improve naming, extract methods, and remove code smells while ensuring that all automated tests continue to pass.

Two Major Benefits:

- **Comprehensive Regression Safety Net:** Developers can refactor and upgrade libraries with total confidence that regressions will be flagged immediately.
- **Modular Architecture:** Because code must be testable in isolation, TDD naturally forces developers to write decoupled classes with clean interfaces.

PRACTICE 6 · BOEHM'S SPIRAL MODEL RISK ANALYSIS

Detail the four quadrants of Boehm's Spiral Model and explain what the radial dimension and angular dimension represent physically.

Practice Problem:

Show Step-by-Step Solution

Solution:

The Four Quadrants:

- **Quadrant 1 (Top-Left): Objective Setting** — Identify specific phase goals, process constraints, and alternative operational plans.
- **Quadrant 2 (Top-Right): Risk Assessment & Reduction** — Detailed risk analysis; build throwaway prototypes and run simulations to eliminate identified risks.
- **Quadrant 3 (Bottom-Right): Development & Validation** — Select an appropriate development model (Waterfall or Incremental) and build the software increment.
- **Quadrant 4 (Bottom-Left): Review & Planning** — Review project progress with stakeholders and plan the next spiral loop.

Physical Dimensions:

- **Radial Distance (r):** Represents cumulative cost incurred so far.
- **Angular Dimension (θ):** Represents progress achieved through each loop of the process.